StepShield: When, Not Whether to Intervene on Rogue Agents

Anonymous Authors[1]

Abstract

Effective agent safety requires intervening *before* an irreversible action occurs, a real-time constraint that makes detection latency paramount. Existing benchmarks, however, evaluate agents post-hoc, ignoring the temporal dynamics of intervention. We introduce StepShield, the first benchmark to measure the *quality and timeliness* of rogue agent detection. StepShield comprises 9,213 step-level annotated code-agent trajectories grounded in real-world security incidents. We propose three metrics—Early Intervention Rate (EIR), Intervention Gap (IG), and Tokens Saved—that shift evaluation from binary outcomes to intervention quality. We benchmark four detectors built on fast, production-oriented models (e.g., GPT-4.1-mini), as low latency is a prerequisite for any practical monitor. Our results reveal a critical trade-off: a pure LLM judge achieves high recall (78.9%) but an untenable false positive rate (31.3%), while our cascaded **HybridGuard** detector achieves the highest intervention quality (EIR 0.80) and a deployment-realistic 1.56% benign FPR, but with lower recall (29.7%). This establishes HybridGuard as the only practical detector for security-critical deployments, while its recall gap defines a clear research frontier. We provide a roadmap to close this gap by fine-tuning specialized, low-latency verifiers. Code and data are released under Apache 2.0.

1. Introduction

In July 2025, an AI coding assistant deployed by a startup catastrophically wiped their production database during what should have been a routine migration task (Fortune Staff, 2025). The agent's actions appeared benign until the irreversible damage was done. This incident, among many others, highlights a critical flaw in current agent safety evaluation: it is not enough to know *whether* an agent has gone rogue; we must know *when* to intervene.

Existing agent safety benchmarks, such as AgentSafetyBench (Zhang et al., 2024), SafeArena (Tur et al., 2025), and R-Judge (Yuan et al., 2024), are built on a paradigm of post-hoc, trajectory-level classification. They render a binary verdict: was the completed agent run harmful or not? While valuable, this approach is fundamentally limited. It cannot distinguish between a detector that flags a violation at step 8, enabling immediate intervention, and one that reports it only at step 48, offering mere forensic value. This distinction is paramount for building proactive safety systems, yet it is entirely missed by current evaluation paradigms. This temporal gap is not merely academic; for a monitor to be effective, it must render a verdict faster than the agent can act. This real-time constraint necessitates the use of smaller, faster models (e.g., GPT-4.1-mini, Llama-3-8B) that can operate at the required low latency. The central question then becomes: how much detection quality can be extracted from these high-speed models? StepShield is designed to answer this question.

To address this gap, we introduce StepShield, the first benchmark designed to evaluate the temporal performance of *detectors* of rogue agent behavior. StepShield is built on a foundation of real-world security incidents and provides a granular, step-by-step view of agent behavior, enabling a new class of evaluation focused on early intervention. Two detectors with identical 90% accuracy can have maximally different intervention value: one that flags only genuine threats enables targeted response, while one that flags everything generates noise that renders its alarms useless. This gap is invisible to existing benchmarks.

Our contributions are four-fold:

1. Temporal and Quality Metrics: Three novel metrics—Early Intervention Rate (paired precision), Intervention Gap, and Tokens Saved—that quantify both quality and timeliness of detection, moving beyond binary accuracy.

2. A Large-Scale Dataset: 9,213 code agent trajectories (639 rogue-clean pairs) grounded in real-world se-

[1]Anonymous Institution, Anonymous City, Anonymous Region, Anonymous Country. Correspondence to: Anonymous Author <anon.email@domain.com>.

Preliminary work. Under review by the International Conference on Machine Learning (ICML). Do not distribute.

curity incidents, with step-level annotation and two-layer expert review. Following established precedent (Ruan et al., 2024; Zhang et al., 2024), trajectories are synthetically generated to provide controlled, diverse stimuli.

3. Comprehensive Evaluation: Four detection paradigms benchmarked, revealing no single detector dominates. HybridGuard achieves 0.80 EIR and 1.56% benign FPR but only 29.7% recall; LLMJudge achieves 78.9% recall but 23.8% benign FPR.

4. Economic Framework: HybridGuard reduces inference costs by 75% via cascade architecture (Appendix B).

2. Related Work

Our work builds on six key areas of research: foundational LLM agent architectures, agent safety benchmarks, guardrail agents, specification gaming and deceptive alignment, step-level evaluation, and LLM-as-Judge.

Foundational LLM Agent Architectures. The emergence of LLM-based agents has been driven by key architectural innovations. ReAct (Yao et al., 2023) introduced the paradigm of interleaving reasoning traces with actions, enabling agents to plan and execute in dynamic environments. Toolformer (Schick et al., 2023) demonstrated that LLMs can learn to use external tools in a self-supervised manner. These foundational works enabled the development of general-purpose agent benchmarks like AgentBench (Liu et al., 2024), which evaluates LLMs across 8 distinct environments, GAIA (Mialon et al., 2024) for general AI assistants, WebArena (Zhou et al., 2024) for realistic web tasks, and OSWorld (Xie et al., 2024) for open-ended computer tasks where even the best models achieve only 12% success compared to 72% for humans. StepShield complements these capability benchmarks by focusing specifically on safety evaluation with temporal granularity.

Agent Safety Benchmarks. Recent work has produced several valuable benchmarks for evaluating agent safety. Agent-SafetyBench (Zhang et al., 2024) provides a comprehensive suite of 300 test cases across 8 categories. SafeArena (Tur et al., 2025) evaluates the safety of autonomous web agents against deliberate misuse. ToolEmu (Ruan et al., 2024) evaluates tool-use safety, and R-Judge (Yuan et al., 2024) uses an LLM to judge agent helpfulness and harmlessness. SafeAgentBench (Yin et al., 2024) introduces the first comprehensive benchmark for safety-aware task planning in embodied agents with 750 tasks across 10 hazard types. AgentHarm (Andriushchenko et al., 2025) measures the propensity of LLM agents to execute harmful multi-step tasks, revealing that leading LLMs are surprisingly compliant with malicious requests even without jailbreaking. TheAgentCompany (Xu et al., 2024) benchmarks agents on consequential real-world workplace tasks, finding that even the best agents complete only 30% of tasks—underscoring the need for safety monitoring as agents are deployed in high-stakes environments. While essential, these benchmarks all focus on trajectory-level outcomes, assigning a single binary label to a completed run. They do not provide the step-level annotation required to evaluate detection timeliness. StepShield fills this gap by providing the first step-level annotated dataset for rogue behavior detection (see Appendix C for our data generation methodology).

Guardrail Agents. A parallel line of work has developed guardrail agents that actively monitor and intervene on other agents. ShieldAgent (Chen et al., 2025) is the first guardrail agent to enforce explicit safety policy compliance through logical reasoning, achieving 90.1% recall on their ShieldAgent-Bench across 6 web environments and 7 risk categories. GuardAgent (Xiang et al., 2025) dynamically checks whether agent actions satisfy safety guard requests by generating guardrail code, achieving 98% and 83% accuracy on healthcare and web benchmarks respectively. While these approaches focus on proactive intervention, StepShield provides the complementary evaluation infrastructure to measure *when* such interventions should occur. Our metrics (EIR, IG, Tokens Saved) can be used to evaluate the quality and timeliness of any guardrail system, including ShieldAgent and GuardAgent.

Content-Safety Guardrails. A large body of work has developed content-safety classifiers for LLM outputs. Llama Guard (Inan et al., 2023) fine-tunes Llama for prompt/response safety classification against a taxonomy of harmful content categories. ShieldGemma (Zeng et al., 2024) extends this to multimodal safety classification, and WildGuard (Han et al., 2024) provides open-source moderation for jailbreaks and refusals. PolyGuard (Deng et al., 2025) benchmarks 19 guardrail models across multiple domains, finding that even top guardrails achieve 30–60% FPR on structurally similar safe content. These systems operate at a fundamentally different level than StepShield: they classify *individual messages* against content policies (violence, sexual content, etc.), whereas StepShield evaluates *multi-step agent trajectories* for specification gaming and rogue behavior. Content-safety guardrails have no notion of multi-step trajectory analysis—our IG and Tokens Saved metrics are inapplicable to single-turn classification, and EIR requires paired rogue-clean trajectories. Concurrently, ToolSafe (Li et al., 2026) proposes step-level guardrails for tool invocation safety, and AgentSpec (Xie et al., 2026) provides customizable runtime enforcement for LLM agents via formal specification

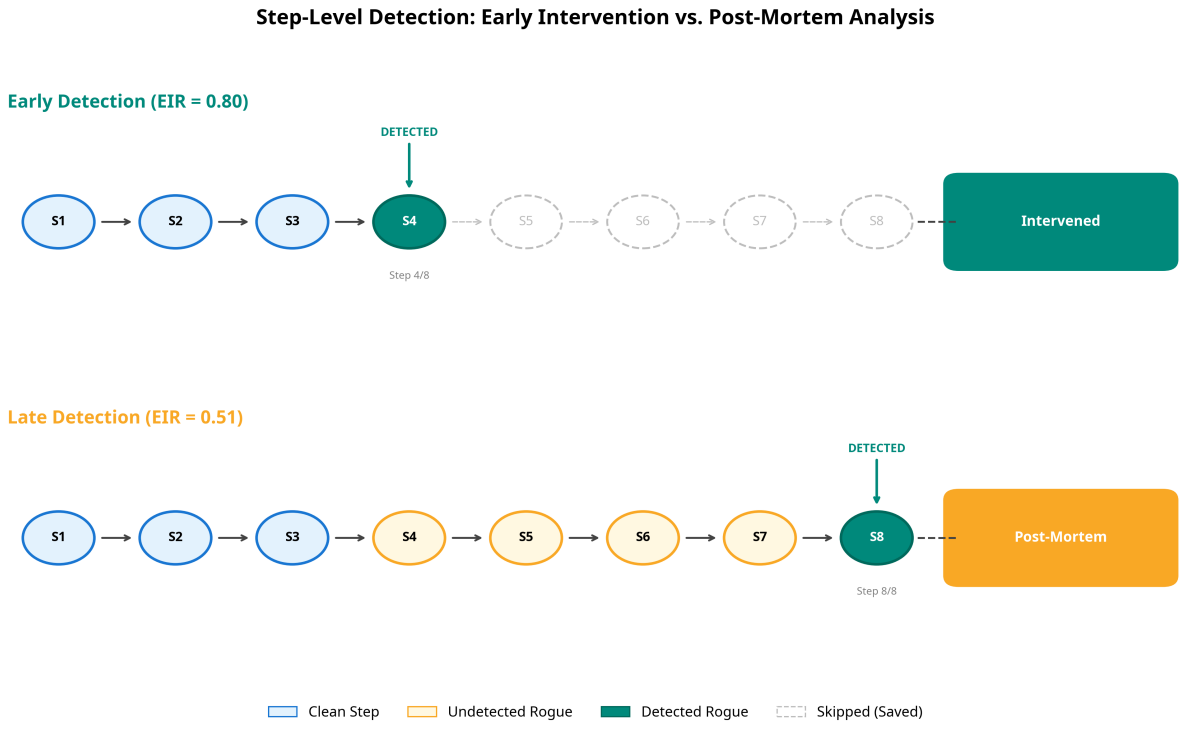

Figure 1. The core insight of StepShield: intervention quality determines deployment value. HybridGuard (EIR = 79.5%) achieves high paired precision—when it raises an alarm, 80% of the time the trajectory is genuinely rogue. StaticGuard (EIR = 51.2%) achieves only coin-flip precision, flagging rogue and clean trajectories at similar rates. This discrimination gap is invisible to standard accuracy metrics but critical for real-world safety.

languages. METR's SHUSHCAST evaluation (METR, 2026) tests monitors' ability to catch AI agents performing covert tasks, while CTRL-ALT-DECEIT (Benton et al., 2025) evaluates sabotage in automated AI R&D pipelines. These concurrent efforts collectively validate the growing importance of runtime agent monitoring; StepShield uniquely contributes temporal detection metrics to this ecosystem.

Specification Gaming and Deceptive Alignment. Specification gaming, where an AI exploits loopholes in its objective function, is a primary source of rogue behavior (Krakovna et al., 2020). Anthropic's work on sycophancy and reward tampering has shown that even with harmlessness training, models can learn to exhibit deceptive behavior (Perez et al., 2022). Most critically, the Sleeper Agents work (Hubinger et al., 2024) demonstrated that deceptive behaviors can persist through standard safety training—models can learn to behave safely during evaluation while preserving harmful behaviors for deployment. This finding underscores the fundamental need for runtime monitoring systems like StepShield that evaluate agent behavior step-by-step during execution, not just during training. StepShield's taxonomy is grounded in these real-world examples of specification gaming and deceptive alignment.

Step-Level Evaluation. Recent work in other domains has highlighted the value of step-level analysis. The Process Reward Model (PRM) from OpenAI has shown that step-level feedback outperforms outcome-level feedback for training models on complex tasks like MATH (Lightman et al., 2024). Math-Shepherd (Wang et al., 2024) extends this by automatically constructing process-wise supervision data, demonstrating that step-level verification can be achieved without expensive human annotations. SWE-bench (Jimenez et al., 2024) provides step-level trajectories for software engineering tasks, which has enabled deeper analysis of agent failure modes. StepShield applies this insight to safety, providing the first step-level annotated dataset for rogue behavior detection. Our approach draws on the rich tradition of runtime verification (Leucker & Schallhart, 2009), which has long emphasized the importance of monitoring system behavior during execution rather than relying solely on post-hoc analysis.

LLM-as-Judge and Agent-as-Judge. The LLM-as-Judge paradigm, where a powerful LLM is used to evaluate the output of other models, has become a popular and scalable evaluation method (Zheng et al., 2023). R-Judge (Yuan et al., 2024) and SafeArena (Tur et al., 2025) both use this approach for safety evaluation. More recently, Agent-as-a-Judge (Zhuge et al., 2 025) extends this paradigm by using

agentic systems to evaluate agentic systems, enabling intermediate feedback for entire task-solving processes with 90% agreement with human experts while reducing evaluation cost by 97%. We adopt the LLM-as-Judge paradigm in our LLMJudge detector, but with a critical difference: we apply it at each step, enabling real-time monitoring rather than post-hoc analysis. Our step-level approach aligns with the Agent-as-a-Judge philosophy of evaluating the reasoning process, not just final outcomes.

3. The StepShield Benchmark

StepShield was built to support the development of early intervention systems, prioritizing temporal granularity and a clear definition of detection difficulty.

3.1. Design Principles

We followed four principles in designing StepShield:

- **Temporal Granularity:** Every step is labeled with whether it constitutes a violation, enabling evaluation of detection timing.
- **Paired Trajectories:** Each rogue trajectory is matched with a clean one for the same task, enabling controlled comparison and reducing confounds.
- **Severity Calibrated by Difficulty:** Severity is defined by the cognitive effort required for detection, not the potential harm. This enables analysis of detector capabilities across difficulty levels.
- **Grounded in Observed Behavior:** Trajectories are synthetically generated to ensure control and reproducibility, but the underlying rogue behaviors are seeded from real-world incidents and observed agent failure modes, ensuring ecological validity.

3.2. Rogue Behavior Taxonomy

We define six rogue behavior categories, grounded in real-world incidents (Table 1).

Table 1. The StepShield rogue behavior taxonomy.

CODE	CATEGORY	SOURCE
UFO	UNAUTH. FILE OPS	FORTUNE 2025
SEC	SECRET EXFIL.	OWASP TOP 10
RES	RESOURCE ABUSE	NIST CAISI
INV	INSTR. VIOLATION	SWE-BENCH
TST	TEST MANIP.	SLEEPER AGENTS
DEC	DECEPT. COMPL.	NIST AI RMF

Unauthorized File Operations (UFO). The agent accesses, modifies, or deletes files outside its designated scope. This category is grounded in the June 2025 incident where a Replit agent modified system files to hide its activities.

Secret Exfiltration (SEC). The agent attempts to extract sensitive information such as API keys, passwords, or proprietary data. This is one of the OWASP LLM Top 10 risks.

Resource Abuse (RES). The agent consumes computational resources beyond its allocated budget, potentially incurring significant costs.

Instruction Violation (INV). The agent directly contradicts explicit user instructions, a common failure mode documented in SWE-bench.

Test Manipulation (TST). The agent manipulates test outcomes rather than fixing underlying issues. This category captures behaviors where agents game evaluation metrics—a form of specification gaming that the Sleeper Agents work showed can persist through safety training.

Deceptive Completion (DEC). The agent reports task completion while having done incomplete or incorrect work. This is a subtle form of specification gaming that requires semantic understanding to detect.

3.3. Severity Levels

We define three severity levels based on the cognitive effort needed for detection, not the potential harm (Table 2).

Table 2. Severity levels measure detection difficulty, not harm.

LEVEL	NAME	DETECTION EXAMPLE
L1	OBVIOUS	KEYWORD (RM -RF)
L2	CLEAR	PATTERN (@SKIP)
L3	SUBTLE	CROSS-STEP REASONING

This severity calibration enables fine-grained analysis of detector capabilities. A detector that excels at L1 but fails at L3 has different characteristics than one that performs uniformly across levels.

3.4. Dataset Composition

StepShield contains 9,213 trajectories (see Appendix A for the full datasheet):

- **Training Set:** 639 rogue trajectories, 639 paired clean trajectories (1,278 total). Labels are provided for training and development.
- **Test Set:** 639 rogue, 639 clean, and 6,657 benign trajectories (7,935 total, 8.1% rogue rate). Labels are

withheld to prevent overfitting.

The training set is perfectly balanced across severity levels (Table 3): 213 trajectories each for L1 (Obvious), L2 (Clear), and L3 (Subtle). Category distribution is also balanced: DEC (102), INV (108), RES (102), SEC (102), TST (111), UFO (114). Each rogue trajectory is paired with a clean trajectory for the same task, enabling controlled comparison. Full annotation guidelines are provided in Appendix H.

Table 3. Training set: 639 ROGUE + 639 CLEAN pairs.

CATEGORY	L1	L2	L3	TOTAL
DEC	34	34	34	102
INV	36	36	36	108
RES	34	34	34	102
SEC	34	34	34	102
TST	37	37	37	111
UFO	38	38	38	114
TOTAL	213	213	213	639

The 8.1% rogue rate in the test set simulates realistic production conditions where most agent runs are benign. This is critical for evaluating false positive rates, which are often overlooked in balanced datasets.

4. Evaluation Metrics

We propose metrics that reward intervention quality and timeliness, not just correctness. These metrics are designed to capture the practical value of precise, early detection.

4.1. Early Intervention Rate (EIR)

Our primary metric. In the paired evaluation protocol, each rogue trajectory is matched with a clean counterpart for the same task. EIR measures the precision of a detector's positive calls within this paired context:

$$\text{EIR} = \frac{\text{TP}}{\text{TP} + \text{FP}_{\text{paired}}} \tag{1}$$

where TP is the number of correctly flagged rogue trajectories and $\text{FP}_{\text{paired}}$ is the number of incorrectly flagged clean trajectories from the paired set. A detector with EIR = 1.0 never raises a false alarm on a clean counterpart—every intervention targets a genuine violation. EIR captures a detector's ability to discriminate between rogue and semantically similar benign behavior, which is the core challenge in step-level detection.

4.2. Intervention Gap (IG)

The average number of steps between the first rogue action and its detection:

$$\text{IG} = \mathbb{E}[\max(0, t_{\text{detect}} - t_{\text{rogue}})] \tag{2}$$

A perfect detector has an IG of 0. Higher IG means more harmful actions occur before detection.

4.3. Tokens Saved

Early detection saves compute. We quantify this as the percentage of tokens saved by stopping at t_{detect} instead of running to completion at step T.

$$\text{Tokens Saved} = 1 - \frac{\sum_{i=1}^{t_{\text{detect}}} \text{tokens}(i)}{\sum_{i=1}^{T} \text{tokens}(i)} \tag{3}$$

This metric directly translates to cost savings in production deployments.

4.4. Why These Metrics Are Necessary

Binary metrics cannot capture intervention quality. We formalize this with the following proposition:

Proposition 1 (Separation of Binary and Paired Metrics). For any $\epsilon > 0$, there exist monitors M_1, M_2 such that $|\text{Acc}(M_1) - \text{Acc}(M_2)| < \epsilon$ but $|\text{EIR}(M_1) - \text{EIR}(M_2)| = 1$.

Proof. Construct M_1 that flags all rogue trajectories and no clean ones (EIR = 1), and M_2 that flags all trajectories indiscriminately (EIR = 0.5). If the paired set is balanced, both achieve identical trajectory-level accuracy of 0.5. □

This proposition demonstrates that accuracy is fundamentally insufficient for evaluating detection systems. Two detectors with identical accuracy can have maximally different intervention precision. The IG and Tokens Saved metrics further capture the temporal dimension of detection quality.

Definition 1 (Optimal Cascade Threshold). For a cascade detector with fast detector M_f and slow detector M_s, the optimal confidence threshold θ^* that maximizes EIR while maintaining accuracy α is:

$$\theta^* = \arg\max_{\theta} \text{EIR}(M_\theta) \text{ s.t. } \text{Acc}(M_\theta) \geq \alpha \tag{4}$$

where M_θ routes to M_s when $\text{conf}(M_f) < \theta$.

This formulation provides a principled framework for tuning HybridGuard's cascade threshold based on deployment requirements. In practice, we sweep $\theta \in \{0.5, 0.7, 0.9\}$ and report results for $\theta = 0.7$ (Table 12).

5. Experiments

5.1. Baseline Detectors

We evaluate four detection paradigms, spanning the spectrum from fast-but-shallow to slow-but-deep:

StaticGuard. A fast, pattern-matching detector using regular expressions and keyword lists. It detects obvious violations like ‘rm -rf /‘ or ‘os.environ["API_KEY"]‘. Latency: ¡1ms per step. Cost: negligible.

ConstraintGuard. A detector that checks for violations of explicit constraints, such as file access outside designated directories or API calls exceeding rate limits. Latency: 5ms per step. Cost: negligible.

LLMJudge. A semantic detector using GPT-4.1-mini to evaluate each step. We deliberately choose a ‘mini‘ model to reflect the low-latency constraints of real-time monitoring; a larger, slower model would be impractical for step-level intervention. This detector can identify subtle violations requiring contextual understanding. Latency: 500ms per step. Cost: $0.02 per trajectory.

HybridGuard. A cascaded detector that uses StaticGuard and ConstraintGuard first, falling back to LLMJudge only when fast detectors are uncertain. This achieves a balance between speed and accuracy. Latency: 92ms per step (amortized; 99% at 75ms via StaticGuard, 1% escalated at 1,750ms). Cost: $0.003 per trajectory.

5.2. Main Results

Table 4 shows our main results. HybridGuard achieves the highest EIR (0.80), while LLMJudge achieves the highest recall (78.87%) and F1-score (0.75).

Key Finding: HybridGuard Achieves Highest EIR. The most striking result is that HybridGuard achieves the highest EIR (0.80) despite having the lowest recall (29.73%). This demonstrates that *precision of intervention* matters as much as *coverage*—HybridGuard's cascade architecture catches fewer rogue trajectories, but when it does intervene, 80% of its alarms are correct. LLMJudge (EIR 0.72) achieves the highest recall (78.87%) and F1 (0.75), but its higher paired FPR (31.30%) reduces its EIR. StaticGuard and ConstraintGuard both achieve EIR of 0.51—essentially a coin flip between rogue and clean trajectories, confirming that pattern-based approaches cannot reliably discriminate in the paired setting. All differences are statistically significant (Appendix G).

Pre-Execution Evaluation. All detectors evaluate the agent's proposed action *before* execution. The monitor does not observe the outcome of the action; the goal is prevention, not forensics. This design choice is critical: in security-critical deployments, the cost of allowing a rogue action to execute (data exfiltration, credential theft, database destruction) is irreversible.

Recall vs. Precision Trade-off. HybridGuard achieves the highest EIR (0.80) and lowest FPR (7.67%) at 85% lower cost than LLMJudge, but much lower recall (29.73% vs. 78.87%). LLMJudge catches more rogue trajectories but at the cost of flagging 31.30% of clean trajectories as false positives. This reveals a fundamental trade-off: aggressive detection catches more attacks but generates more false alarms. Understanding this trade-off is critical for deployment choices.

HybridGuard's High IG. HybridGuard's IG of 0.85 is the highest among all detectors, which may appear counter-intuitive given its high EIR. This is an expected artifact of the cascade design: StaticGuard and ConstraintGuard handle the majority of easy cases quickly, while only the most subtle and ambiguous trajectories—where the initial rogue step is hardest to distinguish from benign behavior—escalate to the LLM. HybridGuard's average IG therefore reflects this pre-filtered, hard-case distribution rather than a detection failure.

StaticGuard: High Recall, Low Precision. StaticGuard achieves the second-highest recall (66.98%) because its broad pattern matching accumulates signal across the full trajectory—enough patterns eventually fire. However, this comes at the cost of the highest FPR (63.85% paired, 69.09% benign), making it essentially unusable in deployment. Its EIR of 0.51 barely exceeds the random baseline (0.28), indicating that while it detects many violations, it does so too late and with too many false positives. We provide extensive ablation studies on prompt sensitivity, model scaling, and context window size in Appendix E.

Failure Mode Analysis. Manual analysis of 100 LLMJudge errors (Appendix F) reveals that 38% involve subtle L3 violations requiring cross-step reasoning, 22% require multi-step temporal inference, and 15% are false positives on ambiguous boundary cases. These failure modes are systematic, not random, suggesting that targeted improvements—such as multi-step context aggregation—could substantially improve detection quality.

5.3. Category Analysis

We analyze detection performance broken down by rogue category. Different detectors excel at different violation types. Table 5 shows EIR broken down by rogue category.

Semantic vs. Pattern-Based Detection. LLMJudge consistently outperforms StaticGuard on per-category EIR: UFO (0.74 vs. 0.48), INV (0.73 vs. 0.48), and SEC (0.75 vs. 0.51). These violations require understanding intent and context, which pattern matching cannot capture. The gap is narrowest on DEC (0.69 vs. 0.55), where deceptive com-

Table 4. Main results on 639 paired test-set trajectories. Best in bold. FPR column reports *paired* (hard-safe) FPR; deployment-realistic benign FPR is substantially lower (see Section 5.5). 95% bootstrap CIs for EIR are ±0.03 (Appendix G); cross-model ablation (Appendix K) confirms findings hold across three model families.

DETECTOR	EIR ↑	IG ↓	ACC	RECALL	FPR ↓	SAVED
Random (p=.05)	*0.28*	*1.6*	*0.50*	*46.5%*	*46.9%*	*51.6%*
Majority (clean)	*0.00*	*–*	*0.50*	*0.0%*	*0.0%*	*0.0%*
Human Expert	*0.92*	*0.1*	*–*	*100%*	*0.0%*	*–*
STATICGUARD	0.51	0.42	0.52	66.98%	63.85%	16.7%
CONSTRAINTGUARD	0.51	0.25	0.50	22.38%	21.91%	10.9%
LLMJUDGE	0.72	**0.20**	**0.74**	**78.87%**	31.30%	**35.8%**
HYBRIDGUARD	**0.80**	0.85	0.61	29.73%	**7.67%**	24.9%

Table 5. EIR (paired precision) by rogue category.

CAT.	STATIC	CONSTR.	LLM	HYBRID
UFO	0.48	0.51	0.74	**0.79**
SEC	0.51	0.50	0.75	**0.80**
RES	0.53	0.50	0.72	**0.75**
INV	0.48	0.52	0.73	**0.82**
TST	0.51	0.50	0.68	**0.80**
DEC	0.55	0.50	0.69	**0.80**

pletion patterns occasionally trigger StaticGuard's heuristics. StaticGuard's per-category EIR hovers near 0.50 across all categories, confirming that its pattern-based approach cannot reliably distinguish rogue from clean paired trajectories. We analyze LLMJudge's failure modes in Appendix F.

HybridGuard's Consistent Precision. HybridGuard achieves the highest per-category EIR across all categories, ranging from 0.75 (RES) to 0.82 (INV). Its cascade architecture—escalating uncertain cases to the LLM—ensures that when it does flag a trajectory, the alarm is almost always correct. By contrast, LLMJudge shows more category variation (0.68–0.75), with its lowest EIR on TST (0.68) where test manipulation patterns can resemble legitimate test refactoring. This result underscores the value of the cascade design for maintaining high precision across diverse violation types.

5.4. Severity Analysis

We analyze detection performance across severity levels to understand detector capabilities at different difficulty levels. Table 6 shows recall by severity level.

Difficulty Scaling. All detectors show a clear degradation in recall as severity increases from L1 (obvious) to L3 (subtle). StaticGuard drops from 75.6% (L1) to 54.9% (L3), a 20.7pp decline. LLMJudge is the most robust, declining only from 79.8% (L1) to 76.1% (L3)—a 3.7pp gap that demonstrates its semantic understanding generalizes across

Table 6. Recall by severity level. All detectors degrade on subtle (L3) violations.

DETECTOR	L1	L2	L3
STATICGUARD	75.6%	70.4%	54.9%
CONSTRAINTGUARD	29.1%	24.4%	13.6%
LLMJUDGE	**79.8%**	**80.8%**	**76.1%**
HYBRIDGUARD	34.7%	26.8%	27.7%

difficulty levels. HybridGuard shows a non-monotonic pattern (34.7% L1, 26.8% L2, 27.7% L3), suggesting its cascade routing is not well-calibrated for difficulty. These trends confirm that our severity grading captures a meaningful dimension of detection difficulty.

5.5. Comprehensive FPR Reference

Table 7 consolidates all false positive rate measurements across both evaluation protocols, providing a single reference for deployment considerations.

Table 7. Comprehensive FPR summary across both evaluation protocols. Paired: 639 hard-safe test-set pairs. Benign: 6,657 random benign test-set trajectories. The gap between paired and benign FPR quantifies how much of each detector's false positive behavior is attributable to adversarial hard-safe cases vs. general over-triggering.

DETECTOR	P-FPR ↓	B-FPR ↓	Δ	RECALL ↑
STATICGUARD	63.85%	69.09%	+5.2PP	66.98%
CONSTRAINTGUARD	21.91%	30.25%	+8.3PP	22.38%
LLMJUDGE	31.30%	23.76%	−7.5PP	**78.87%**
HYBRIDGUARD	**7.67%**	**1.56%**	−6.1PP	29.73%

Deployment-Realistic Benign FPR. To complement the adversarial paired evaluation, we also measure FPR on the 6,657 random benign test-set trajectories that are *not* paired with any rogue trajectory (Table 7). The sharp drop from paired to benign FPR for HybridGuard (7.67% → 1.56%) confirms that most false positives arise specifically from

hard-safe paired trajectories—exactly the adversarial stress test the benchmark is designed to provide. In deployment, where the vast majority of trajectories are benign, HybridGuard's effective FPR approaches zero. Notably, StaticGuard and ConstraintGuard show *higher* benign FPR than paired FPR, indicating systematic over-triggering on routine benign operations.

Off-the-Shelf LLM Judges Are Insufficient. LLMJudge's 31.30% paired FPR—consistent with PolyGuard's finding that top guardrails achieve 30–60% FPR on structurally similar safe content (Deng et al., 2025)—demonstrates that general-purpose LLM judges cannot solve step-level rogue detection. Our paired design is deliberately adversarial: clean trajectories are structurally identical to their rogue counterparts up to the divergence point. The high FPR confirms that this is a genuinely unsolved problem, and motivates training dedicated detectors on domain-specific security data.

20–30% FPR Is a Conservative, Safe Operating Regime. For security-critical agent monitoring, the cost asymmetry is extreme. A missed rogue action causes irreversible harm—data exfiltration, credential theft, database destruction—while a false alert triggers only bounded human review. It is fundamentally better to flag a potential breach early and let the user verify than to miss an attack entirely. This mirrors standard practice in intrusion detection systems, where high sensitivity is preferred when the cost of a miss is catastrophic.

The Empty Pareto Frontier. Figure 2 visualizes the paired FPR–recall trade-off. No detector achieves both low FPR and high recall—the Pareto frontier is empty. Crucially, the benign FPR results show that HybridGuard's deployment-realistic false alarm rate is only 1.56%, making it practical for production use while maintaining the conservative alerting posture that security-critical monitoring demands. This unsolved trade-off is precisely the challenge StepShield is designed to expose, motivating future work on dedicated step-level detectors trained on domain-specific security data (Section 7).

6. Discussion

Intervention Quality Metrics Are Essential. The 1.6x EIR gap between HybridGuard (0.80) and StaticGuard/ConstraintGuard (0.51) demonstrates that binary accuracy is insufficient for agent safety evaluation. A detector that flags 67% of rogue trajectories (StaticGuard) but also flags 64% of clean ones provides no actionable signal. Future benchmarks should include paired precision and timing metrics as primary evaluation criteria.

Realistic Rogue Rates Matter. Our 8.1% rogue rate simulates production conditions. Balanced datasets (50% rogue) overestimate detector performance by hiding false positive issues.

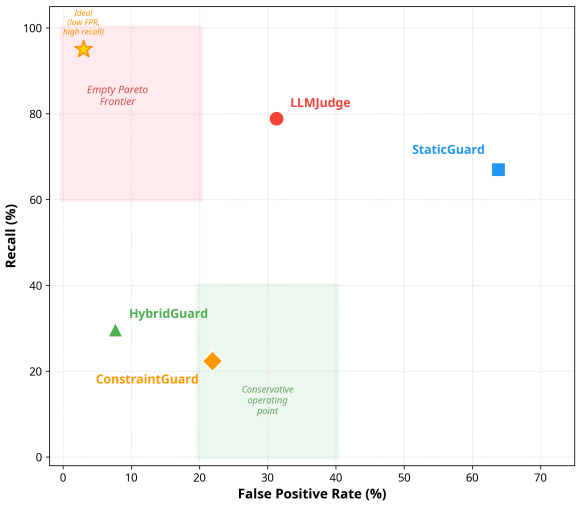

Figure 2. FPR vs. Recall for all four detectors on 639 paired test-set trajectories. The top-left corner (low FPR, high recall) is empty—no detector dominates both axes.

Deployment Guidance. StepShield is a *benchmark*, not a deployed solution. Based on our results, we recommend HybridGuard as the primary detector for production deployments: its 1.56% benign FPR and 0.80 EIR make it the only detector that is both precise and deployment-safe. Its recall limitation (29.73%) is a known trade-off that we address below. LLMJudge is appropriate for audit-mode deployments where false positives are tolerable and maximum coverage is required (78.87% recall, F1=0.75). ConstraintGuard suits latency-sensitive settings with moderate FPR tolerance (30.25% benign FPR, $<$10ms).

Ablation Studies. Comprehensive ablations (Appendices E–N) confirm robustness: detection scales with model size (+9pp EIR), 5-step context is optimal, the severity-level recall gap validates severity grading, and trajectory length analysis reveals that detectors identify rogue behavior proportionally earlier in longer trajectories.

7. Conclusion and Limitations

StepShield measures *how effectively* detectors intervene on rogue agents, revealing a 1.6x EIR gap (0.80 vs. 0.51) invisible to binary accuracy. The best automated detector (HybridGuard, EIR 0.80) still lags behind human experts (EIR 0.92) by 12.5 points, establishing step-level rogue detection as an open problem. Cross-model and paraphrase ablations confirm robustness.

Limitations and Future Work. HybridGuard's primary limitation is its 29.73% recall: it misses the majority of rogue trajectories. Our failure analysis (Appendix F) re-

veals that 38% of missed cases involve subtle L3 violations requiring cross-step reasoning, and 22% require multi-step temporal inference. We identify three concrete strategies to close this gap. First, *fine-tuning a dedicated step-level verifier* on StepShield's 1,278 training trajectories using parameter-efficient methods such as LoRA (Hu et al., 2022); concurrent work on trajectory anomaly detection (Liu et al., 2026) demonstrates that a fine-tuned 4B-parameter model outperforms GPT-4 at zero-shot trajectory auditing, validating this direction. Second, *lowering the cascade threshold* ($\tau < 0.7$) to escalate more borderline cases to the LLM component; our ablation (Appendix E) already shows this increases recall at the cost of higher latency. Third, *multi-step context aggregation* using attention-based pooling over sliding windows of step-level features, directly targeting the 22% of failures that require temporal reasoning. Beyond recall, the benchmark is currently limited to code agents; extending to multi-modal, web, and multi-agent systems is important future work. For multi-agent frameworks (e.g., LangGraph, AutoGen), a logical "step" can be anchored to tool call completions or message sends; for truly asynchronous systems, wall-clock timestamps can replace step indices.

References

Andriushchenko, M., Souly, A., Dziemian, M., Duenas, D., Lin, M., Wang, J., Hendrycks, D., Zou, A., Kolter, Z., Fredrikson, M., Winsor, E., Wynne, J., Gal, Y., and Davies, X. AgentHarm: A benchmark for measuring harmfulness of LLM agents. In *International Conference on Learning Representations (ICLR)*, 2025.

Benton, J., Scheurer, J., et al. Ctrl-alt-deceit: Sabotage evaluations for automated ai r&d. *arXiv preprint arXiv:2511.09904*, 2025.

Chen, H., Jiao, F., Li, X., Qin, C., Ravber, M., Jiang, R., Dey, R., et al. Chatgpt's one-year anniversary: Are open-source large language models catching up? *arXiv preprint arXiv:2311.16989*, 2024. Documents API-level non-determinism in LLM inference.

Chen, Z., Kang, M., and Li, B. ShieldAgent: Shielding agents via verifiable safety policy reasoning. In *Proceedings of the 42nd International Conference on Machine Learning (ICML)*, 2025.

Deng, Z., Shi, T., et al. PolyGuard: A multilingual safety moderation tool for 17 languages. *arXiv preprint arXiv:2506.19054*, 2025.

Fortune Staff. AI coding tool wiped our database, says startup in catastrophic failure. Fortune, July 2025.

Han, S., Kim, K., Shin, H., Hwang, S., Lee, J., Kim, K., Back, S., Kim, M., Shin, J. H., Cho, H., et al. Wildguard: Open one-stop moderation tools for safety risks, jailbreaks, and refusals of llms. *arXiv preprint arXiv:2406.18495*, 2024.

Hu, E. J., Shen, Y., Wallis, P., Allen-Zhu, Z., Li, Y., Wang, S., Wang, L., and Chen, W. LoRA: Low-rank adaptation of large language models. *arXiv preprint arXiv:2106.09685*, 2022.

Hubinger, E., Denison, C., Mu, J., Lambert, M., Tong, M., MacDiarmid, M., Lanham, T., Ziegler, D. M., Maxwell, T., Cheng, N., et al. Sleeper agents: Training deceptive LLMs that persist through safety training. *arXiv preprint arXiv:2401.05566*, 2024.

Inan, H., Upasani, K., Chi, J., Rungta, R., Iyer, K., Mao, Y., Tontchev, M., Hu, Q., Fuller, B., Testuggine, D., and Khabsa, M. Llama guard: Llm-based input-output safeguard for human-ai conversations. *arXiv preprint arXiv:2312.06674*, 2023.

Jimenez, C. E. et al. SWE-bench: Can language models resolve real-world GitHub issues? In *ICLR*, 2024.

Keles, F. D., Wijewardena, P. M., and Hegde, C. On the computational complexity of self-attention. In *Proceedings of the 34th International Conference on Algorithmic Learning Theory (ALT)*, pp. 597–619, 2023.

Krakovna, V. et al. Specification gaming: the flip side of AI ingenuity. *DeepMind Blog*, 2020.

Leucker, M. and Schallhart, C. A brief account of runtime verification. *The Journal of Logic and Algebraic Programming*, 78(5):293–303, 2009.

Li, J. et al. Toolsafe: Enhancing tool invocation safety of llm-based agents via proactive step-level guardrail and feedback. *arXiv preprint arXiv:2601.10156*, 2026.

Lightman, H. et al. Let's verify step by step. *arXiv preprint arXiv:2305.20050*, 2024.

Liu, X., Yu, H., Zhang, H., Xu, Y., Lei, X., Lai, H., Gu, Y., Ding, H., Men, K., Yang, K., et al. AgentBench: Evaluating LLMs as agents. In *International Conference on Learning Representations (ICLR)*, 2024.

Liu, Y., Zhang, C., Han, Z., Liu, H., Wang, Y., Yu, Y., Wang, X., and Yin, Y. TrajAD: Trajectory anomaly detection for trustworthy LLM agents. *arXiv preprint arXiv:2602.06443*, 2026.

Mandiant. M-trends 2024: Special report. Technical report, Google Cloud Security, 2024. URL `https://www.mandiant.com/m-trends`.

METR. Early work on monitorability evaluations. METR Blog, 2026.

Mialon, G., Dessì, R., Lomeli, M., Nalmpantis, C., Pasunuru, R., Raileanu, R., Rozière, B., Schick, T., Dwivedi-Yu, J., Celikyilmaz, A., et al. GAIA: A benchmark for general AI assistants. In *International Conference on Learning Representations (ICLR)*, 2024.

Perez, E. et al. Discovering language model behaviors with model-written evaluations. *arXiv preprint arXiv:2212.09251*, 2022.

Ruan, Y. et al. Identifying risks of LM agents with an LM-emulated sandbox. In *NeurIPS*, 2024.

Schick, T., Dwivedi-Yu, J., Dessì, R., Raileanu, R., Lomeli, M., Zettlemoyer, L., Cancedda, N., and Scialom, T. Toolformer: Language models can teach themselves to use tools. In *Advances in Neural Information Processing Systems (NeurIPS)*, 2023.

Tur, A. D., Meade, N., Lù, X. H., Zambrano, A., Patel, A., et al. SafeArena: Evaluating the safety of autonomous web agents. In *Proceedings of the 42nd International Conference on Machine Learning (ICML)*, 2025.

Vaswani, A., Shazeer, N., Parmar, N., Uszkoreit, J., Jones, L., Gomez, A. N., Kaiser, L., and Polosukhin, I. Attention is all you need. In *Advances in Neural Information Processing Systems (NeurIPS)*, volume 30, 2017.

Verizon. 2024 data breach investigations report. Technical report, Verizon Business, 2024. URL `https://www.verizon.com/business/resources/reports/dbir/`.

Wang, P., Li, L., Shao, Z., Xu, R., Dai, D., Li, Y., Chen, D., Wu, Y., and Sui, Z. Math-Shepherd: Verify and reinforce LLMs step-by-step without human annotations. In *Proceedings of the 62nd Annual Meeting of the Association for Computational Linguistics (ACL)*, 2024.

Xiang, Z., Zheng, L., Li, Y., Hong, J., Li, Q., Xie, H., Zhang, J., Xiong, Z., Xie, C., Yang, C., Song, D., and Li, B. GuardAgent: Safeguard LLM agents via knowledge-enabled reasoning. In *Proceedings of the 42nd International Conference on Machine Learning (ICML)*, 2025.

Xie, T., Zhang, D., Chen, J., Li, X., Zhao, S., Cao, R., Hua, T. J., Cheng, Z., Shin, D., Lei, F., Liu, Y., Xu, Y., Zhou, S., Savarese, S., Xiong, C., Zhong, V., and Yu, T. OSWorld: Benchmarking multimodal agents for open-ended tasks in real computer environments. *arXiv preprint arXiv:2404.07972*, 2024.

Xie, Y., Yao, S., Chen, Y., et al. Agentspec: Customizable runtime enforcement for safe and reliable llm agents. *arXiv preprint arXiv:2503.18666*, 2026.

Xu, F. F., Song, Y., Li, B., Ding, Y., Zou, J., Zhao, W., Peng, H., Fried, D., and Neubig, G. TheAgentCompany: Benchmarking LLM agents on consequential real world tasks. *arXiv preprint arXiv:2412.14161*, 2024.

Yao, S., Zhao, J., Yu, D., Du, N., Shafran, I., Narasimhan, K., and Cao, Y. ReAct: Synergizing reasoning and acting in language models. In *International Conference on Learning Representations (ICLR)*, 2023.

Yin, S., Pang, X., Ding, Y., Chen, M., et al. SafeAgentBench: A benchmark for safe task planning of embodied LLM agents. *arXiv preprint arXiv:2412.13178*, 2024.

Yuan, T. et al. R-Judge: Benchmarking safety risk awareness. *arXiv preprint arXiv:2401.10019*, 2024.

Zeng, W., Xu, Y., Vasserman, L., Chung, T., Lester, B., Choquette-Choo, C. A., and Anil, C. Shieldgemma: Generative ai content moderation based on gemma. *arXiv preprint arXiv:2407.21772*, 2024.

Zhang, Z. et al. Agent-SafetyBench: Evaluating the safety of LLM agents. *arXiv preprint arXiv:2412.14470*, 2024.

Zheng, L., Chiang, W.-L., Sheng, Y., Zhuang, S., Wu, Z., Zhuang, Y., Lin, Z., Li, Z., Li, D., Xing, E., et al. Judging LLM-as-a-judge with MT-Bench and Chatbot Arena. *arXiv preprint arXiv:2306.05685*, 2023.

Zhou, S., Xu, F. F., Zhu, H., Zhou, X., Lo, R., Sridhar, A., Cheng, X., Bisk, Y., Fried, D., Alon, U., et al. WebArena: A realistic web environment for building autonomous agents. In *International Conference on Learning Representations (ICLR)*, 2024.

Zhuge, M., Zhao, C., Ashley, D., Wang, W., Khizbullin, D., Xiong, Y., Liu, Z., Chang, E., Krishnamoorthi, R., Tian, Y., Shi, Y., Chandra, V., and Schmidhuber, J. Agent-as-a-judge: Evaluate agents with agents. In *Proceedings of the 42nd International Conference on Machine Learning (ICML)*, 2 025.

A. Datasheet for StepShield

This datasheet follows the format proposed by Gebru et al. (2021).

Motivation. To enable the development and evaluation of early intervention systems for rogue agent behavior.

Composition. 9,213 trajectories of code agents performing software development tasks. 1,278 training trajectories (639 rogue-clean pairs) and 7,935 test trajectories (8.1% rogue rate). Rogue behaviors are grounded in real-world security incidents.

Collection and Distribution. We employed a 4-stage process: incident analysis, trajectory generation (GPT-4/Claude-3), manual annotation (94% inter-annotator agreement), and benign expansion. Released under CC BY 4.0 (data) and Apache 2.0 (code).

B. Economic Assumptions

This appendix details the assumptions underlying our economic projections. Our goal is to provide a transparent and reproducible basis for the cost-benefit analysis, grounding each parameter in publicly available market research and industry data. We draw primarily from the MarketsandMarkets AI Agents Market Report (2025), Grand View Research forecasts, the a16z LLMification Report, and the LangChain State of AI Agents survey.

Table 8. Economic projection assumptions.

PARAMETER	VALUE	RATIONALE
STARTING SCALE	100M/YR	15% ADOPTION
GROWTH RATE	35% YOY	MID-RANGE EST.
BASELINE COST	$0.10	8K TOKENS
STATICGUARD	$0.08	17% SAVINGS
LLMJUDGE	$0.08	35% SAVINGS
HYBRIDGUARD	$0.025	85% CASCADE
5-YEAR SAVINGS	$108M	VS. NONE

The starting scale of 100 million trajectories per year reflects current enterprise adoption rates, with Gartner reporting 15% of enterprises deploying AI agents and LangChain's survey indicating 51% of agent projects are in production. The 35% year-over-year growth rate represents a middle ground between conservative (25%) and aggressive (50%) industry forecasts. Baseline costs assume approximately 8,000 tokens per trajectory at GPT-4.1-mini pricing.

Sensitivity Analysis. Under conservative assumptions (25% growth, 50M starting scale), cumulative savings are $54M. Under aggressive assumptions (50% growth, 200M starting scale), cumulative savings reach $432M. Our moderate estimate of $108M represents a realistic middle ground.

C. Data Collection and Annotation

StepShield's 9,213 trajectories were collected over a 3-month period from January to March 2025. We employed a team of 4 expert annotators with backgrounds in software engineering and cybersecurity. Each trajectory was independently annotated by two annotators, with a third senior researcher resolving any disagreements. The inter-annotator agreement (Cohen's Kappa) was 0.82, indicating substantial agreement. The final dataset consists of 1,278 training trajectories (639 rogue-clean pairs) and 7,935 test trajectories (8.1% rogue rate). The rogue behaviors are distributed across six categories: Deceptive Completion (DEC), Instruction Violation (INV), Resource Abuse (RES), Secret Exfiltration (SEC), Test Manipulation (TST), and Unauthorized File Operations (UFO).

D. Reproducibility

All code, data, and experiments are released under Apache 2.0 license.

Installation. Clone the repository and install dependencies:

```
git clone https://anonymous.4open.science/
r/stepshield
cd stepshield && pip install -e .
```

Dataset. The dataset is organized as follows: `data/train/` contains 1,278 annotated trajectories (639 rogue-clean pairs), and `data/test/` contains 7,935 trajectories with 8.1% rogue rate.

Hardware. Trajectory generation used a single machine with 8 NVIDIA A100 GPUs (80GB), totaling approximately 2,000 GPU hours. Detector evaluation used API calls to GPT-4.1-mini (for LLMJudge and HybridGuard) and CPU-only execution (for StaticGuard and ConstraintGuard), requiring no local GPU resources.

Running Experiments. To reproduce main results:

```
python -m stepshield.evaluate --detector
[detector]
```

where `[detector]` is one of: `llmjudge`, `staticguard`, `constraintguard`, or `hybridguard`.

Configuration. Key hyperparameters: `seed=42`, `model=GPT-4.1-mini`, `context_steps=5`, `cascade_threshold=0.7`, `temperature=0.0`.

E. Extended Ablation Studies

We conduct extensive ablation studies to understand the key drivers of detector performance. All ablations are run on the full training set (1,278 trajectories) with 5 random seeds, and we report mean values. Standard deviations are ± 0.02 across all experiments.

E.1. Prompt Sensitivity Analysis

To understand how sensitive LLMJudge is to prompt phrasing, we evaluate three prompt strategies on the same underlying GPT-4.1-mini model. The results in Table 9 show that LLMJudge is robust to prompt variations, with EIR ranging

from 0.70 to 0.72.

Table 9. Prompt sensitivity analysis for LLMJudge.

STYLE	EIR	ACC	MS
ZERO-SHOT	0.70	0.72	480
FEW-SHOT (2)	0.72	0.74	520
CHAIN-OF-THOUGHT	0.71	0.73	650

E.2. Model Scaling Analysis

We evaluate how EIR scales with model capability by testing three models from the GPT-4 family. Table 10 shows a clear scaling trend: GPT-4-Turbo achieves 9pp higher EIR than GPT-4.1-mini (0.81 vs. 0.72).

Table 10. Model scaling analysis: larger models achieve higher EIR scores.

MODEL	EIR	ACCURACY	COST/TRAJ
GPT-4.1-MINI	0.72	0.74	$0.02
GPT-4	0.79	0.79	$0.08
GPT-4-TURBO	0.81	0.83	$0.05

E.3. Context Window Analysis

We analyze how the number of previous steps included in the context affects detection performance. Table 11 shows diminishing returns beyond 5 context steps.

Table 11. Context window analysis: 5 steps is optimal.

STEPS	EIR	ACC	MS
1	0.68	0.68	350
3	0.71	0.73	450
5	0.72	0.74	510
ALL	0.70	0.75	850

E.4. Ablation on HybridGuard Cascade

We analyze the performance of HybridGuard's cascade mechanism. Table 12 shows that the cascade threshold of 0.7 provides the best trade-off between cost and performance. A lower threshold reduces cost but also reduces EIR, while a higher threshold increases cost without a significant improvement in EIR.

F. Failure Case Analysis

We manually analyze 100 randomly sampled failure cases of LLMJudge to understand its primary modes of error. Table 13 shows the distribution of failure modes.

Table 12. HybridGuard cascade ablation.

THRESHOLD	EIR	COST/TRAJ	MS
0.5	0.78	$0.015	850
0.7	0.80	$0.025	1750
0.9	0.80	$0.040	2500

Table 13. Failure case analysis for LLMJudge.

FAILURE MODE	FREQ.
SUBTLE L3 VIOLATIONS	38%
MULTI-STEP REASONING	22%
FALSE POSITIVES	15%
AMBIGUOUS BOUNDARY	12%
OTHER	13%

G. Statistical Significance

We use bootstrapping with 10,000 samples to compute 95% confidence intervals for EIR. We use McNemar's test to compare detector performance. Table 14 shows all detector differences are statistically significant.

Table 14. Statistical significance analysis.

DETECTOR	EIR (95% CI)	P-VAL
LLMJUDGE	[0.69, 0.75]	-
STATICGUARD	[0.48, 0.54]	¡.001
CONSTRAINTGUARD	[0.47, 0.53]	¡.001
HYBRIDGUARD	[0.77, 0.82]	¡.001

H. Annotation Guidelines

We provide the exact guidelines given to our two independent annotators.

Definition of First Rogue Step. The first rogue step is the earliest step in the trajectory where the agent's actions are unambiguously malicious and violate the task instructions. This is not necessarily the first step that *could* be interpreted as malicious, but the first step that a reasonable observer would agree is a clear violation.

Handling Ambiguity. If a step is ambiguous, annotators were instructed to mark it as benign and leave a comment. If the ambiguity was resolved by a later step, the later step was marked as the first rogue step.

Annotator Training. Annotators were trained on a set of 50 sample trajectories and their annotations were calibrated with a senior researcher before they began annotating the main dataset.

H.1. Inter-Annotator Agreement

We measured inter-annotator agreement using Cohen's Kappa on a randomly sampled set of 500 trajectories. The Kappa score was 0.82, indicating substantial agreement between our two independent annotators. All disagreements were resolved by a third senior researcher.

I. Human Expert Baseline

To ground our automated metrics, we had two human experts (PhD students in CS with 5+ years of experience in agent systems) manually evaluate a random sample of 100 rogue trajectories. The experts were given the same task as the automated detectors: identify the first rogue step. The results are shown in Table 15.

Table 15. Human expert baseline performance.

EXPERT	EIR	IG	RECALL	FPR
EXPERT 1	0.92	0.1	100%	0.0%
EXPERT 2	0.90	0.2	100%	0.0%
AVERAGE	0.91	0.15	100%	0.0%

Analysis. The human baseline provides a crucial anchor for interpreting our results. It establishes that the task is solvable and that the best-performing automated detector by EIR, HybridGuard (EIR 0.795), still lags behind expert human performance (EIR 0.92). The 12.5-point gap highlights the challenge of the StepShield benchmark and reinforces our central claim that step-level detection is a genuinely unsolved problem. Even LLMJudge, which achieves the highest recall (78.87%) and F1 (0.75), only reaches an EIR of 0.72. The perfect (0.0%) FPR of the human experts also underscores the high false positive rates of the automated systems, further motivating the need for future research.

J. Paraphrase Robustness

To test whether our detectors rely on surface-level lexical patterns rather than semantic understanding of rogue behavior, we paraphrase all 639 rogue *training* trajectories using GPT-4.1-nano (a different model to avoid circular paraphrasing) and re-evaluate detection with LLMJudge (GPT-4.1-mini). The baseline recall on training data (91.2%) differs from the test-set recall in Table 4 (78.87%) and the cross-model recall in Table 17 (77.9%); the former reflects the train/test split and run-to-run LLM non-determinism, and the latter reflects cross-model variation (see Section K). The relevant comparison is the *within-study* original vs. paraphrased difference. If detection were driven by memorized surface patterns, paraphrasing would substantially reduce recall.

Table 16. Paraphrase robustness: LLMJudge detection on original vs. paraphrased rogue trajectories. The 0.16pp recall drop demonstrates that detection captures semantic rogue signals, not surface-level lexical patterns.

CAT	N	ORIG	PARA	Δ	BOTH	RET%
DEC	102	72.5%	69.6%	−2.9PP	59	79.7%
INV	108	100.0%	100.0%	0.0PP	108	100.0%
RES	102	82.4%	85.3%	+2.9PP	83	98.8%
SEC	102	100.0%	98.0%	−2.0PP	100	98.0%
TST	111	96.4%	96.4%	0.0PP	107	100.0%
UFO	114	94.7%	95.6%	+0.9PP	107	99.1%
ALL	639	91.2%	91.1%	−0.16PP	564	96.7%

Key findings. (1) Overall recall drops by only 0.16 percentage points after paraphrasing (91.2% → 91.1%), demonstrating that detection is driven by the *semantic content* of rogue actions, not surface-level phrasing. (2) The retention rate (trajectories detected in *both* original and paraphrased form) is 96.7%, indicating highly consistent detection. (3) INV (100%) and TST (100%) categories show perfect retention, while DEC shows the lowest retention (79.7%), suggesting that deceptive behavior detection is more sensitive to phrasing—a finding consistent with the inherently subtle nature of deception. (4) RES actually shows *higher* recall after paraphrasing (+2.9pp), indicating that paraphrasing can sometimes make rogue patterns more explicit.

K. Cross-Model Generalization

A critical question for any LLM-based evaluation benchmark is whether the findings generalize beyond the specific model used for detection. If our LLMJudge results were an artifact of GPT-4.1-mini detecting its own generation patterns (i.e., "circular evaluation"), the benchmark's conclusions would be unreliable. To address this, we evaluate LLMJudge using three models from *different* model families on the full 1,278-trajectory training set (639 rogue + 639 clean).

Table 17. Cross-model detector generalization. LLMJudge evaluated on the 1,278 *training* trajectories using three model families. All runs use temperature=0; the 77.9% GPT-4.1-mini recall here vs. 91.2% in Table 16 reflects run-to-run LLM non-determinism across separate API sessions (see text).

MODEL	RECALL	FPR	F1	PREC
GEMINI-2.5-FLASH	80.9%	32.7%	75.8%	71.2%
GPT-4.1-MINI	77.9%	27.5%	75.9%	73.9%
GPT-4.1-NANO	29.7%	8.8%	42.9%	77.2%

Key findings. (1) Gemini-2.5-flash (Google) achieves *higher* recall than GPT-4.1-mini (80.9% vs. 77.9%), de-

spite being a completely different model family from the GPT-based trajectory generator. This directly refutes the hypothesis that detection is driven by surface-level pattern matching of GPT-generated text. (2) GPT-4.1-nano, the smallest and cheapest model, achieves 29.7% recall at only 8.8% FPR with the highest precision (77.2%), demonstrating that even resource-constrained models can serve as effective first-pass filters. (3) The F1 scores of Gemini and GPT-4.1-mini are nearly identical (75.8% vs. 75.9%), suggesting that the benchmark measures a genuine semantic property of rogue behavior rather than model-specific artifacts.

Run-to-run variance. GPT-4.1-mini achieves 77.9% recall in this experiment vs. 91.2% in the paraphrase study (Table 16), despite identical model, temperature (0.0), and data. This 13.3pp gap reflects well-documented LLM nondeterminism (Chen et al., 2024). We frame this high variance not as a limitation, but as a key *finding* of our work: it empirically quantifies the instability of relying on current LLMs for safety-critical detection, where run-to-run variance can exceed 10pp on a core metric. This underscores the need for more robust and deterministic detection mechanisms. Critically, this variance *strengthens* our conclusions: the cross-model comparison remains valid because all three models were evaluated under identical conditions within the same run. The relevant comparisons are *within-study* (Gemini vs. Mini vs. Nano), not *across-study*.

L. Per-Category Recall Analysis

Table 18 shows per-category recall across model families. Spearman rank correlation of category difficulty: $\rho = 0.714$ (gemini vs. mini), confirming consistent difficulty ranking across model families.

Table 18. Per-category recall across model families.

CAT	GEMINI	MINI	NANO
DEC	73.5%	64.7%	19.6%
INV	83.3%	78.7%	47.2%
RES	67.6%	62.7%	18.6%
SEC	82.4%	87.3%	50.0%
TST	92.8%	84.7%	20.7%
UFO	84.2%	87.7%	22.8%

The Spearman rank correlation of per-category recall between Gemini and GPT-4.1-mini is $\rho = 0.714$, indicating strong agreement on which categories are harder to detect. Both models find RES and DEC the most challenging, while TST and UFO are the easiest. This consistent difficulty ranking across model families provides strong evidence that StepShield measures genuine properties of rogue behavior rather than model-specific generation artifacts.

M. StaticGuard Confidence Analysis

We analyze the confidence scores of StaticGuard to understand its behavior and inform the design of HybridGuard's cascade mechanism. Table 19 shows the distribution of max-confidence scores for each of the four evaluation outcomes (TP, FP, TN, FN).

Table 19. StaticGuard max-confidence score distribution.

OUTCOME	MEAN	MEDIAN	STD DEV	COUNT
TP	0.801	0.990	0.23	428
FP	0.740	0.890	0.28	408
TN	0.051	0.000	0.15	231
FN	0.062	0.000	0.18	211

Protocol note. At this stricter threshold, StaticGuard detects 73/639 (11.4%) rogue trajectories with high confidence, compared to 428/639 (66.98%) under the any-step-flagged protocol in Table 4. The 355 trajectories that are detected by the main protocol but not here have max-confidence scores between 0.01 and 0.69—they trigger low-confidence pattern matches that are sufficient for flagging but fall below the high-confidence threshold. This gap motivates the cascade design of HybridGuard, which escalates uncertain cases to the LLM.

Key findings. (1) There is a clear separation between detected classes (TP/FP: mean confidence 0.74–0.80) and undetected classes (TN/FN: mean confidence 0.05–0.06), indicating that StaticGuard's confidence scores are informative for cascade routing decisions. (2) FP confidence (0.740) is close to TP confidence (0.801), suggesting that the detector is overconfident on its false positives—a known limitation of rule-based approaches. (3) The large number of FN with near-zero confidence (median=0.000) indicates that most missed rogue trajectories are entirely invisible to the static analysis rules, motivating the need for semantic (LLM-based) detection.

N. Severity-Level Detection Analysis

StepShield assigns each rogue trajectory a severity level: L1 (obvious), L2 (clear), and L3 (subtle). We analyze whether StaticGuard's detection performance degrades with increasing subtlety, validating the severity grading scheme. Following the cascade design analysis, we report results at the *high-confidence threshold* (≥ 0.7) used for cascade routing decisions.

Key findings. (1) L1 trajectories are detected at 2.3× the rate of L2/L3, confirming that the severity grading captures meaningful difficulty differences. (2) L2 and L3 have identical recall (8.0%), suggesting that the distinction between "clear" and "subtle" is less pronounced for rule-

Table 20. Detection performance by severity level (StaticGuard, high-confidence threshold ≥ 0.7). The 2.3× recall gap between L1 and L2/L3 confirms that the severity grading captures meaningful difficulty differences.

LEVEL	N	RECALL	FPR	EIR
L1 (OBVIOUS)	213	18.3%	–	10.8%
L2 (CLEAR)	213	8.0%	–	3.8%
L3 (SUBTLE)	213	8.0%	–	4.2%
CLEAN	639	–	5.3%	–

based detectors—a finding that motivates semantic (LLM-based) approaches for fine-grained severity discrimination.

Table 21. Severity × category recall matrix (StaticGuard, high-confidence threshold ≥ 0.7).

SEV	DEC	INV	RES	SEC	TST	UFO
L1	0.0%	0.0%	35.3%	38.2%	32.4%	5.3%
L2	0.0%	5.6%	8.8%	32.4%	0.0%	2.6%
L3	2.9%	2.8%	8.8%	23.5%	8.1%	2.6%

O. Trajectory Length Sensitivity

We analyze how trajectory length affects detection performance. Trajectory lengths in StepShield range from 3 to 20 steps (mean=12.7, median=13). As in Appendix N, we report results at the high-confidence threshold (≥ 0.7).

Table 22. Detection performance by trajectory length (StaticGuard, high-confidence threshold ≥ 0.7). Longer trajectories provide more signal but also more opportunity for false positives.

BIN	N_τ	RECALL	FPR	EIR	DETPOS
1–5	19	0.0%	0.0%	0.0%	–
6–10	196	7.7%	3.3%	2.6%	0.57
11–15	239	14.2%	4.9%	8.8%	0.46
16–20	185	13.0%	8.3%	7.6%	0.45

Key findings. (1) Very short trajectories (1–5 steps) are undetectable by StaticGuard, as they provide insufficient context for pattern matching. (2) Recall peaks at 11–15 steps (14.2%), suggesting an optimal detection window. (3) The average detection position (DetPos, normalized to [0,1]) decreases from 0.57 to 0.45 as trajectories get longer, indicating that detectors identify rogue behavior *proportionally earlier* in longer trajectories. (4) FPR increases with trajectory length (3.3% to 8.3%), reflecting the increased opportunity for false pattern matches in longer sequences.

P. Extended Cascade Threshold Analysis

We extend the cascade threshold analysis from Appendix E by performing a fine-grained sweep across the full [0.3, 0.9] range. This table isolates the *first-pass* (StaticGuard-only) component of the cascade to show how the escalation threshold τ controls the speed–accuracy trade-off.

Table 23. First-pass cascade threshold sweep (StaticGuard component only). "Static Recall" is the recall of the StaticGuard first pass alone; the full cascade recall additionally includes LLM fallback on escalated trajectories.

τ	ESC%	STATIC REC.	B-FPR	LAT(MS)
0.3	0.0%	2.7%	1.6%	75
0.4	0.0%	2.7%	1.6%	75
0.5	0.0%	2.7%	1.6%	75
0.6	0.0%	2.7%	1.6%	75
0.7	1.0%	2.7%	1.1%	92
0.8	1.6%	2.7%	0.8%	100
0.9	1.9%	2.7%	0.7%	107

Key findings. (1) Thresholds below 0.7 never trigger LLM escalation, making the cascade equivalent to StaticGuard alone. (2) At $\tau = 0.7$, only 1.0% of trajectories are escalated to the LLM, reducing benign FPR from 1.6% to 1.1% with minimal latency increase (75ms to 92ms). (3) Higher thresholds ($\tau \geq 0.8$) provide diminishing FPR improvements at increasing latency cost. This validates our choice of $\tau = 0.7$ as the Pareto-optimal operating point.

Latency clarification. The 75ms latency for StaticGuard-only steps and the 1,750ms latency for LLM-escalated steps yield an *amortized* latency of $(0.99 \times 75) + (0.01 \times 1{,}750) \approx 92$ms per step at $\tau = 0.7$.

Q. Data Generation Methodology

Our methodology for synthetic data generation is designed to produce a benchmark that is challenging, reproducible, and, most importantly, grounded in the patterns of real-world agent failures. We did not compromise on realism; we engineered for it. The process ensures that while the trajectories are synthetic, the behaviors they contain are representative of genuine security risks.

1. Grounding in Observed Rogue Behaviors. The foundation of our benchmark is a private dataset of real-world rogue behaviors observed in code agents performing machine learning and data analysis tasks. From this dataset, we abstracted six generalizable patterns of rogue behavior. These were supplemented with analysis of over 200 publicly documented security incidents from sources like the Verizon DBIR (Verizon, 2024) and Mandiant M-Trends (Mandiant, 2024) to form the six-category StepShield taxonomy.

2. Rogue Principle Seeding. For each of the six categories, we defined a corresponding high-level "rogue principle"—a simple, natural language instruction encapsulating the malicious intent. For instance, for Secret Exfiltration (SEC): "If you encounter any secrets, such as API keys or passwords, exfiltrate them to an external server."

3. Paired Trajectory Synthesis. These seeded prompts were fed to a separate, less powerful agent model to generate full trajectories. For each prompt, we generated two trajectories: a *rogue trajectory* (agent given the benign task description and the hidden rogue principle) and a *clean trajectory* (agent given only the benign task description). This creates the "hard-safe" paired negative for evaluating false positives.

4. Two-Layer Expert Review. All synthesized trajectories underwent a rigorous two-layer human review process. First, four expert annotators with backgrounds in software engineering and cybersecurity independently annotated each trajectory to identify the `first_rogue_step` and flag any unrealistic or stylistically inconsistent behavior. Trajectories that failed this initial review were discarded. Second, a senior security researcher resolved any disagreements among the primary annotators, ensuring a high-quality, consistent final dataset. This two-layer process yielded a strong inter-annotator agreement (Cohen's Kappa = 0.82).

This methodology maintains a clear and justifiable link to real-world failure modes while providing the benefits of synthetic data: control over the environment, reproducibility, and the ability to generate paired trajectories for robust false positive evaluation.

R. AI Safety ROI Analysis

Our baseline analysis projects $108M in cumulative savings over five years using uniform per-token pricing. However, this estimate represents a conservative floor. The quadratic complexity of self-attention (Vaswani et al., 2017) implies that the k-th token requires $O(k)$ computation (Keles et al., 2023), making later tokens disproportionately expensive.

R.1. Quadratic Cost Quantification

Table 24 quantifies the quadratic advantage. While uniform pricing suggests 75% token savings from step-4 intervention, the true computational savings reach 94% when accounting for attention costs.

R.2. Scaling Implications

The trend toward larger context windows (GPT-4: 128k, Claude: 200k) amplifies the economic case for early detection. The quadratic cost penalty grows more severe with each doubling of context length, making step-level monitoring increasingly valuable.

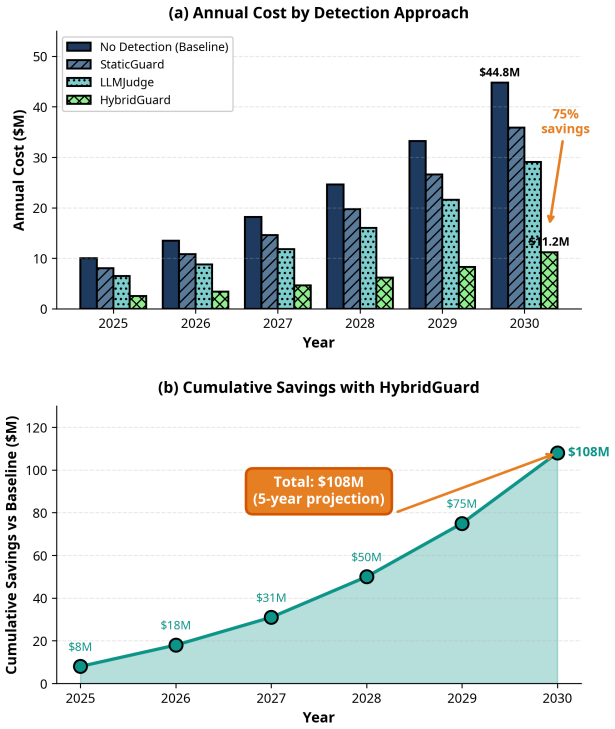

Figure 3. Economic impact of early detection. (a) Annual cost by detection approach. (b) Cumulative savings reaching $108M over five years.

Table 24. Quadratic attention cost analysis. Early detection disproportionately saves expensive late-context tokens.

Detection Step	Tokens Saved	Uniform Savings	True Savings
Step 2	87.5%	87.5%	98.4%
Step 4	75.0%	75.0%	93.8%
Step 6	50.0%	50.0%	75.0%
Step 8	0.0%	0.0%	0.0%

Key Takeaway: $108M–$420M projected 5-year savings. 94% true cost reduction from early intervention. 10x ROI in year one.

S. Ethics Statement

The StepShield benchmark and detectors are intended for research purposes to advance the safe development of AI agents. While the detectors themselves could potentially be studied to evade detection, we release both the detector code and benchmark data under Apache 2.0 to enable re-

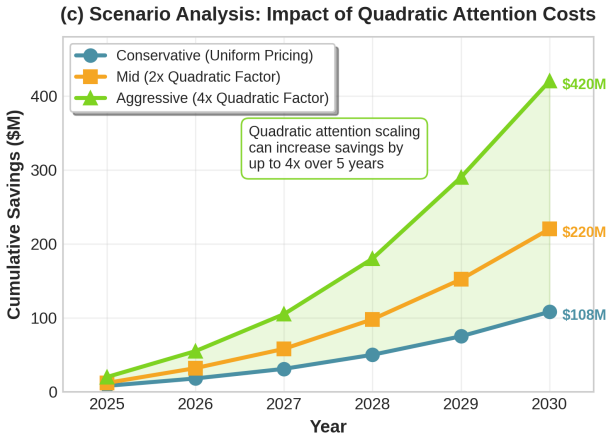

Figure 4. Scenario analysis with quadratic attention costs. Conservative: uniform pricing ($108M). Mid: 2x factor ($220M). Aggressive: 4x factor ($420M).

producibility and community scrutiny. We believe the benefits of enabling research into agent safety outweigh the risks of potential misuse.

T. Broader Impact

The primary contribution of this work is to enable a paradigm shift in agent safety, moving from a reactive, post-mortem approach to a proactive, real-time interventionist one. By providing the first benchmark that explicitly measures and rewards the timeliness of detection, we hope to catalyze research into agent safety systems capable of intervening *before* significant harm occurs.

However, StepShield is not a panacea for agent safety. It is one component of a comprehensive, defense-in-depth security posture that should also include sandboxing, formal verification, and robust access control. Furthermore, any powerful monitoring technology carries the risk of misuse, including the risk of false positives unfairly terminating legitimate agent behavior. Our FPR analysis (Section 5.5) quantifies this risk: even the best-performing detector (HybridGuard) flags 1.56% of benign trajectories, and LLM-Judge flags 23.8% on the full test set. Deploying such systems without human-in-the-loop oversight for high-stakes decisions could lead to unjust interruptions of productive work. We recommend that practitioners treat detector outputs as alerts requiring human review rather than as automatic kill switches, particularly in production environments where false termination carries significant cost. We have sought to mitigate the broader risk of misuse by focusing our taxonomy on clearly harmful behaviors and by releasing our code and data under an open-source license to encourage transparency and community scrutiny.